Autonomous Object Sorting System

Robot Perception and Manipulation (RPM) — Course Project Report

IIIT Delhi · IRAS Hub Lab

Team: Priyanshu Yadav · Abhinay Prakash · Yash Kumar Saini · Ayush Kitnawat · Vinay K. Verma

GitHub: vermavinay982/Autonomous-Object-Perception-Manipulation

Demo Video: Google Drive Link

1. Abstract

This report presents the design, implementation, and evaluation of a fully autonomous robotic object-sorting system built on the UFactory xArm Lite6 manipulator. The system detects, classifies, picks, and places tabletop cubes into designated bins using an explicit perception–action pipeline — without any human intervention during execution. Objects are sorted by two primary visual attributes: color (HSV-based) and visual marker presence (AprilTag/ArUco detection). The system achieved the target sorting success rate of ≥80%, zero manual interventions during demo runs, a YOLO mAP50 of 0.995, and spatial coordinate mapping error under 5 mm.

2. Introduction

Autonomous robotic manipulation is a fundamental challenge in robotics, requiring tight integration of perception, planning, and actuation. This project addresses the problem of tabletop object sorting — a task that demands reliable object detection, attribute classification, coordinate mapping, and pick-and-place execution in an unstructured environment.

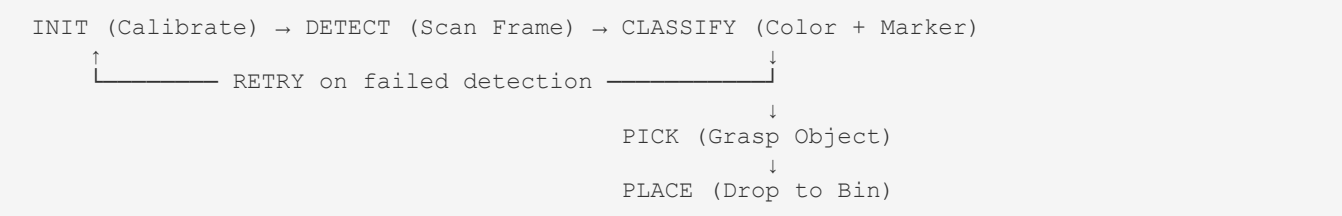
The primary objective was not maximal perceptual sophistication but reliable system integration: every subsystem must work together predictably, with failures detected and handled programmatically. The system was built around an explicit, interpretable pipeline with no black-box end-to-end controllers, ensuring every decision can be traced and explained.

Sorting categories supported:

- blue_plain — blue cube, no marker
- green_marked — green cube, with AprilTag
- red_marked — red cube, with AprilTag

3. System Architecture

The system follows a four-stage Perceive → Classify → Map → Act loop that runs continuously until the workspace is cleared.



3.1 Hardware Stack

The xArm Lite6 is controlled via the xArm Python SDK in Servo Cartesian mode at 10 Hz command rate. A dead-zone and low-pass filter smoothing strategy is applied to all motion commands, with protective-stop fault recovery.

Component	Details
Robotic Arm	UFactory xArm Lite6 (6-DOF)
Gripper	Custom dual-servo gripper via ESP32 MCU
Camera	Intel RealSense RGB-D
Controller	Python SDK over Ethernet
Host	HP laptop running perception + control

3.2 Software Stack

Module	Technology
Object Detection	YOLOv8 (fine-tuned) + OpenCV
Color Classification	HSV thresholding
Marker Detection	ArUco / AprilTag
Coordinate Mapping	Homography + Kabsch Algorithm
Calibration Storage	YAML cache files
Task Logic	Python state machine
Gripper Control	ESP32 via socket (Wi-Fi)

4. Perception Pipeline

4.1 Object Detection

The detection pipeline combines two complementary approaches:

YOLOv8 (primary detector): A YOLOv8 model was fine-tuned on a custom dataset of tabletop cubes. It provides real-time bounding boxes at 10–16 FPS. A workspace polygon boundary filter (defined by AprilTag corners) eliminates detections outside the valid region. Object centroids are extracted from bounding boxes for grasp targeting.

OpenCV HSV Thresholding (color classification): Each detected object's bounding box crop is analyzed in HSV color space. Predefined HSV ranges for blue, green, and red are applied to determine the dominant color attribute. This classical method acts as a reliable, deterministic classifier that does not depend on model confidence alone.

4.2 Marker Detection

ArUco/AprilTag markers are detected in each frame. The presence or absence of a detected marker within an object's bounding box determines the marked vs plain secondary attribute. Marker IDs also serve a dual role as workspace localization anchors (see Section 5).

4.3 Classification Logic

Each object receives a composite label from two attributes:

```
color  ∈ {blue, green, red}
marker ∈ {plain, marked}
→ label ∈ {blue_plain, green_marked, red_marked}
→ bin   ∈ {Bin A, Bin B}
```

A classification confidence score is computed from the HSV thresholding coverage ratio within the bounding box. Objects with ambiguous confidence are subject to retry logic before bin assignment is finalized.

4.4 ML Performance

Three YOLO model variants were evaluated:

Metric	Multi-Class	Multi-Label	YOLOv11n
Precision	0.995	0.997	0.9985
Recall	1.000	0.969	1.000
mAP50	0.995	0.987	0.995
mAP50-95	0.933	—	0.8473
F1 Score	—	0.982	—
Inference	203 ms	—	3.24 ms ✓

The YOLOv11n model was selected for final deployment due to its 3.24 ms inference time with no loss in detection quality (mAP50 = 0.995), enabling smooth real-time operation.

5. Calibration & Coordinate Mapping

Accurate pixel-to-robot coordinate mapping is the critical link between perception and manipulation. The full pipeline is:

```
Camera Frame → HSV/YOLO Detect → AprilTag Homography → Kabsch Transform → xArm XYZ
Command
```

5.1 Intrinsic Calibration

The RealSense camera's intrinsic parameters (focal length, principal point, distortion coefficients) were calibrated using the standard OpenCV checkerboard method. Validated visually by overlaying reprojected points on the image.

5.2 AprilTag Workspace Localization (Extrinsic)

Four AprilTag markers are placed at the known corners of the robot workspace. Each frame, their detected pixel positions are used to compute a live homography matrix mapping pixel coordinates to a flat workspace plane in metric units. This provides robust, drift-free workspace localization that self-corrects every frame.

Occlusion handling: When the robot arm occludes one or more tags during motion, the system falls back to YAML-cached tag poses from the last known good frame. This prevents coordinate pipeline failure mid-execution.

5.3 Kabsch Algorithm (SVD-Based Hand-Eye Calibration)

The Kabsch Algorithm computes the optimal rigid-body transformation (rotation matrix R and translation vector t) between corresponding camera-frame XYZ points and robot-frame XYZ points, using Singular Value Decomposition (SVD). This hand-eye calibration was validated across the full workspace, achieving < 5 mm spatial error — sufficient for reliable cube grasping.

YAML fallback cache: Calibration results are stored in YAML files, preventing the need for full recalibration at every session start while allowing manual refresh when hardware is repositioned.

6. Manipulation Pipeline

6.1 Grasp Strategy

All grasps are top-down with a fixed gripper orientation, simplifying the grasp planning problem significantly. The grasp sequence for each object is:

- Move arm to home position
- Move to pre-grasp waypoint (50 mm above object centroid XY)
- Descend to grasp height (object surface Z)
- Close gripper
- Lift to post-grasp waypoint (50 mm above)
- Move to bin approach pose
- Open gripper (drop object into bin)
- Return to home

Stationary lock: Before initiating a grasp, the system verifies that the object has not moved between two consecutive frames. This prevents grasping a disturbed or rolling object.

6.2 Smart Z-Axis Stacking

As cubes accumulate in a bin, the drop height is incremented by +25 mm per placed object. The bin cube count is tracked in a YAML-cached state, persisting across retry cycles. This prevents cubes from colliding on drop and destabilizing the stack.

6.3 Bin Configuration

- Bin A and Bin B are separated by > 200 mm to prevent misplacement
- Fixed predefined place poses in robot frame
- No tight placement tolerances required

6.4 Fault Recovery & Retry Logic

- If an object is not detected as lifted after gripper close, the arm retries the grasp at least once
- After one failed retry, the object is skipped and the pipeline moves to the next detected object
- The pipeline runs continuously in a loop until no objects remain in the workspace
- All fault recovery is programmatic — no manual intervention is permitted during a demo run

7. Hardware Integration — ESP32 Servo Gripper

The custom gripper uses two servo motors controlled wirelessly by an ESP32 microcontroller communicating with the host PC over Wi-Fi sockets.

Engineering challenges solved:

- Servo gear slipping: Sudden large angle commands caused gear wear. Solved by implementing smooth incremental angle steps instead of direct target jumps.
- PWM jitter: Added motion smoothing and dead-band filtering to reduce vibration and mechanical shock.
- Over-rotation protection: Applied servo angle limits and speed restrictions in firmware.
- Socket communication bug: Diagnosed and resolved a packet-drop bug in the ESP32 ↔ host socket layer that was causing dropped gripper commands and pipeline freezes. The fix restored stable, deterministic real-time communication.

8. Real-World Challenges & Solutions

Challenge	Root Cause	Solution
Arm self-collision on direct paths	Joint limits violated on straight-line Cartesian moves	Added intermediate waypoint above object; arm resets to home between picks
Cubes colliding in same bin	Fixed drop height regardless of stack size	Smart stacking: +25 mm drop_z per cube in bin
Camera occlusion by arm	Tags hidden during arm motion	YAML-backed coordinate cache; fallback to last known tag pose
Calibration drift across sessions	Minor hardware repositioning	Kabsch Algorithm + YAML persistent calibration; < 5 mm residual error
Servo gear wear & slipping	Sudden angle jumps in servo commands	Smooth incremental control, angle limits, speed caps on ESP32
Socket packets dropped	Buffer overrun in ESP32 socket handler	Fixed root cause; restored stable real-time communication

9. Results & Evaluation

Metric	Result	Target
Sort Success Rate	≥ 80%	≥ 80% ✅
YOLO mAP50	0.995	—
YOLO Inference Time	3.24 ms	—
Spatial Coordinate Error	< 5 mm	—
Manual Interventions (demo)	0	0 ✅

The system met all primary project success criteria: it sorted ≥ 80% of objects correctly, ran fully autonomously without human intervention, detected and handled grasp failures programmatically, and all system behavior is fully explainable through the explicit pipeline.

10. Scope for Improvement

- Servo durability: High-speed operation still risks long-term gear backlash. Upgrade to metal-gear or smart servos (e.g., Dynamixel) for better precision and lifespan.
- Force feedback: Add force/torque sensors for adaptive, safer gripping — especially useful for fragile or irregularly shaped objects.
- IK-based motion planning: Integrate MoveIt for full inverse-kinematics planning to eliminate self-collision risks on complex trajectories.
- Depth-based localization: Upgrade to full RGB-D point-cloud processing for 3D object localization, enabling handling of non-flat workspaces and stacked objects.
- VLA integration: Use Vision–Language–Action models (e.g., CLIP, LLaVA) as an overlay for resolving ambiguous classifications, compared against the classical HSV baseline.
- More object categories: Extend the attribute space to include shape (circle vs rectangle) for finer-grained sorting.

11. Conclusion

This project successfully delivered a fully autonomous tabletop object sorting system on the xArm Lite6, exceeding the ≥ 80% sorting success target with zero manual interventions. The system is built on a robust, explicit perception–action pipeline with no black-box dependencies — every decision in the pipeline is interpretable and traceable.

Key technical innovations include: live AprilTag homography workspace mapping, smart Z-axis stacking for bin management, YAML-backed occlusion recovery during arm motion, and stable ESP32 servo control with PWM smoothing. The architecture provides a solid foundation for future extensions including depth sensing, IK-based motion planning, and VLA-based semantic reasoning.

12. References

- UFactory xArm Python SDK Documentation
- OpenCV Camera Calibration and 3D Reconstruction
- Jocher, G. et al. — YOLOv8 (Ultralytics)
- Wang et al. — YOLOv11 (Ultralytics)
- Olson, E. — AprilTag: A robust and flexible visual fiducial system, ICRA 2011
- Kabsch, W. — A solution for the best rotation to relate two sets of vectors, Acta Crystallographica 1976
- Intel RealSense SDK Documentation
- Project Repository: https://github.com/vermavinay982/Autonomous-Object-Perception-Manipulation

Acknowledgements

Special Thanks: Rishav & Rahul (gripper design + camera mounting) · Prof. Jainendra Sir (guidance) · IRAS Hub (robotic platforms)